# Creating an age-depth model with stratigraphic correlation using StratoBayes

**GEOL50215**

## Introduction

We are using the same Permian-Triassic sulphur isotope data as in the correlation worksheet. This time, we are expanding the model in multiple ways: 1) we subdivide the sections into parts that may have different sedimentation rates 2) we add an unconformity to both sections.

## Reading the data

Start by reading the `d34S-Staithes-and-North-Sea.csv` file from the `data` folder, or by setting your working directory to the correct folder on your computer using `setwd`.

```
# read data set used in example 1
d34S <- read.csv("data/d34S-Staithes-and-North-Sea.csv")
```

## Partitioning data into different parts (e.g., lithologies)

To subdivide the sections into subsections, we need a dataframe indicating the top boundaries of the subsections. The subsections could e.g. be lithological units or stratigraphic intervals.

The dataframe needs to have the column names:

- `lith`: the lithological unit or other subsection
- `depth`: the upper boundary of the corresponding unit
- `site`: the name of the site to which this unit belongs

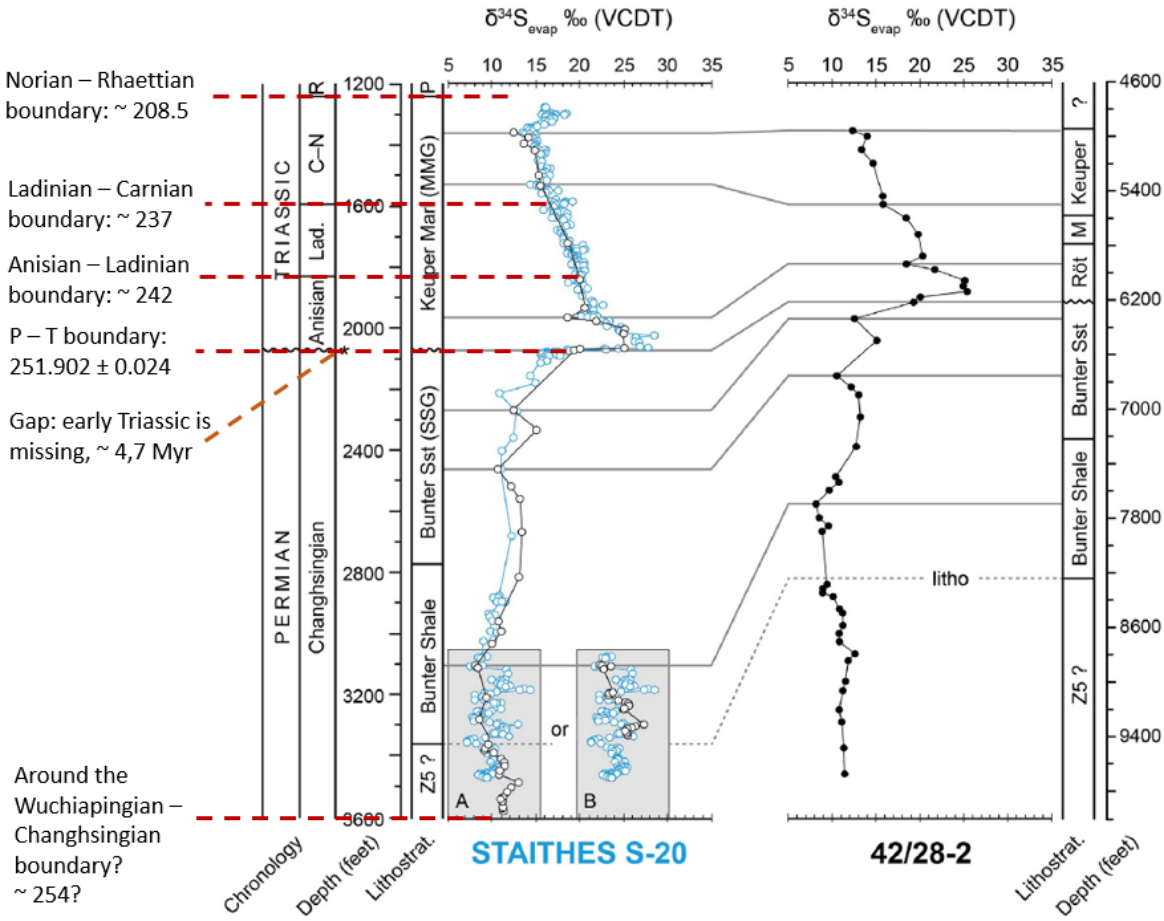



Figure 1: Age constraints from Staithes, and correlation with North Sea well 42/28-2, modified from Salisbury et al. 2023

For including an unconformity (gap), add an extra row with depth equal to the position of the gap, and `lith = "gap"`.

**Step 1: Create a dataframe with lithological or stratigraphic units for the Staithes section**

Start with just a few units, to keep the model simple.

**Hint**

Create a data.frame with the following structure:

```
partsStaithes <- data.frame(partition = ,
                            depth = ,
                            site = )
```

Have a look at Figure 1 Populate that data.frame with a few subsections, possibly representing stratigraphic units or lithologies. The `partition` column holds the name of the subsections, the `depth` column holds the minimal depth associated with that subsection, and `site` should be "Staithes".

**Solution**

```
partsStaithes <- data.frame(partition = c("Changhsingian", "gap",
                                          "Anisian-Lad", "C-N"),
                            depth = c(2080, 2080, 1600, 1200),
                            site = rep("Staithes", 4))
```

We divided the Staithes section into 3 parts corresponding to stratigraphic intervals (stages): "Changhsingian", "Anisian-Lad", and "C-N". We also added an unconformity between the Changhsingian and the Anisian, as is indicated in Figure 1 .

**Step 2: Create a dataframe with lithological or stratigraphic units for the North Sea section (well 42/28-2)**

Then, join the two dataframes. Optionally, if you used depths in feet, convert depths to meters (1 foot = 0.3048 m)

Hint

You can use the same data.frame structure as for the Staithes section, but using the depths in the "North Sea" section this time. To join the two data.frames, you might use `rbind()`.

Solution

```
partsNorthSea <- data.frame(depth = c(8200, 6200, 6200, 5800, 4600),
                            partition = c("Z5", "Bunter", "gap", "Roet",
                                          "M-Keuper"),
                            site = rep("North Sea", 5))

# combine data.frames
lith <- rbind(partsNorthSea, partsStaithes)
# transform depth measurements from feet to m
lith$depth <- lith$depth * 0.3048
```

We used lithostratigraphic units to divide the North Sea section, combining the "Bunter Shale" and "Bunter Sst" units, and also combining the "M" and "Keuper" units. Again, we added an unconformity between the "Bunter" and the "Roet" units.

## Adding ties (absolute age information)

To align the sections on an absolute age scale, rather than a depth scale, we use tie points that associate an age with a specific depth in a section.

The `ties` dataframe needs to have the column names:

- `depth`: the upper boundary of the tie (`numeric`)
- `site`: the name of the site to which this unit belongs (`character`)
- `densityFun`: the name of an R density function such as `dnorm` or `dunif` (`character`)
- `arg1`: the value of the first argument of the density function beyond `x`. For example, if the density function is `dnorm`, `arg 1` correspond to the value of the mean of the normal distribution.

if the density function requires more than one argument beyond `x`, additional `arg2`, `arg3`,… can be added: - `arg2`: the value of the second argument of the density function beyond `x`. For example, if the density function is `dnorm`, `arg2` correspond to the value of the standard deviation (sd) of the normal distribution.

Ideally, these ties would correspond for example to radiometric dates measured in the section. As we don't have those in the data set, we use age estimates from stage boundaries obtained by an earlier correlation of the Staithes section with a composite sulphur isotope section by Salisbury et al. 2022, and look up the age of the stage boundaries in the International Chronostratigraphic Chart.

*Caution: This approach assumes that the correlation of Salisbury et al. 2022 accurately aligns the Staithes section to the boundaries, which probably underestimates the age uncertainty associated with the Staithes section.*

Step 3: Create a dataframe with ties (absolute age information) to anchor the Staithes section in the absolute age scale

For stage boundary ages with an uncertainty estimate, we can use that uncertainty as the standard deviation and the age estimate as the mean. For ages without uncertainty, we can chose a standard deviation that we think reasonably reflects the uncertainty.

Hint

We need a data.frame with the columns `depth`, `site`, `densityFun`, `arg1` and probably `arg2`. If we use normal distributions for the ties (absolute ages), we can use the probability density function "dnorm". If we check what this function does with `?dnorm`, we find that we need to specify a mean and standard deviation to use it. The `arg1` and `arg2` columns then take the mean age (`arg1`) and standard deviation of that age estimate (`arg2`). You can assign ties (absolute ages) to both sections or e.g. just to the "Staithes" section and let the sulphur isotopes take care of the correlation of with the "North Sea" section. The more ties we have, the better constrained the sedimentation rate estimates and positions of the sections on the age scale will be.

```
ties <- data.frame(
  depth = ,
  site = ,
  densityFun = , # "dnorm" for normal distributions
  arg1 = , # means of the normal distributions
  arg2 = , # standard deviations of the normal distributions
)
# do you also need to transform depths to meters?
```

Check Figure 1 for where you might place ties (absolute ages).

Solution

```r
tiesStaithes <- data.frame(
  depth = c(3650, 2081, 1830, 1600, 1250),
  densityFun = rep("dnorm", 5),            # density of normal distribution
  arg1 = c(254, 251.902, 242, 237, 208.5), # means
  arg2 = c(0.5, 0.024, 0.5, 0.5, 0.5),     # standard deviations
  site = rep("Staithes", 5)
)

ties <- rbind(tiesStaithes)
# transform depth measurements from feet to m
ties$depth <- ties$depth * 0.3048
```

We've used the stage boundaries indicated in Figure 1 and defined normal density functions with means corresponding to their mean ages and standard deviations either given by the International Chronostratigraphic Chart, or in most cases, where we didn't have that, we used an arbitrary 0.5 Myr for the standard deviation.

## Reading the data with StratoBayes

Step 4: Read the data using `StratData()`

Hint

First, load the StratoBayes library (if it is not loaded already), using the `library` function. Then, create a `StratData` object with the `StratData()` function. Make sure to specify the name of the `signalColumn`, the `referenceSite`, the `ties` argument, the `lith` argument, and the `heightScale` (`heightScale = "depth"` will multiply depths with -1, and negative depths (or positive heights) are necessary for the package working as intended).

Solution

```
# load StratoBayes library
library("StratoBayes")
# read data with the StratData function
stratDat <- StratData(signal = d34S, signalColumn = c("d34S"),
                      referenceSite = "Staithes", zScale = "depth",
                      zColumn = "depth", ties = ties, parts = lith)
```

## Plotting the data

Step 5: Visualise the subsections (lithological or stratigraphic partitions) of the dataset using the `plot.StratData()` function.

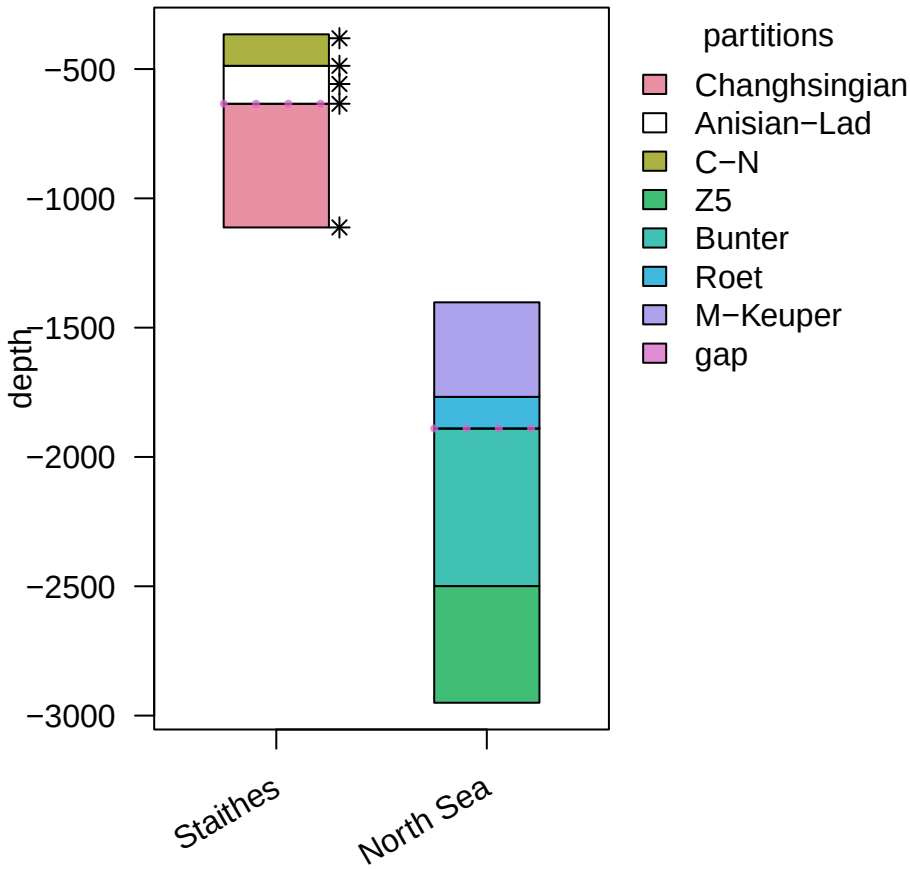
Hint

Check the documentation using `?plot.StratData` to find out what you can specify for the `show` argument.

Solution

The `plot()` function recognises that `stratDat` is an object of class `StratData` and plots it. To show the subsections or lithology partitions, we specify the argument `show = "partition"`.

```
plot(stratDat, show = "partition")
```




The coloured blocks correspond to the subsections we specified. Asterisks mark the positions of the ties. Gaps are shown as white dashed lines.
If you want to see the sulphur isotope data, use `plot(stratDat, show = "signal")`

## Specifying priors

Our dataset has absolute age estimates for at least one section, so we want to do the correlation on the age scale. We also want different sedimentation rates for different lithologies (subsections).

Step 6: Get a template for the priors using the `StratModelTemplate()` function.

Hint

Use `?StratModelTemplate` to find out which `scale` and `sedModel` arguments might be appropriate. You can also change probability distributions using the `alpha`, `gammaLog` or `gap` arguments.

Solution

Specifying that we want the model to run on the "age" scale, we want different sedimentation rates for lithologies (subsections), and a normal distribution as the prior for gaps:

```
StratModelTemplate(
  stratData = stratDat,
  alignmentScale = "age",
  sedModel = "partition",
  gap = "normal"
)
```

```
priors <- structure(list(
  "alpha_Staithes" = UniformPrior(min = , max = ),
  "alpha_North Sea" = UniformPrior(min = , max = ),
  "gammaLog_Changhsingian" = NormalPrior(mean = , sd = ),
  "gammaLog_gap" = NormalPrior(mean = , sd = ),
  "gammaLog_Anisian-Lad" = NormalPrior(mean = , sd = ),
  "gammaLog_C-N" = NormalPrior(mean = , sd = ),
  "gammaLog_Z5" = NormalPrior(mean = , sd = ),
  "gammaLog_Bunter" = NormalPrior(mean = , sd = ),
  "gammaLog_Roet" = NormalPrior(mean = , sd = ),
  "gammaLog_M-Keuper" = NormalPrior(mean = , sd = )),
  class = c("StratPrior", "list"))

model <- StratModel(stratData = stratDat,
                    priors = priors,
                    alignmentScale = "age",
                    sedModel = "partition",
                    alphaPosition = "middle",
                    nKnots = 25)

result <- RunStratModel(stratObject = stratDat,
                        stratModel = model,
```

```
                        nRun = 1,
                        nIter = 1000)

`stratData` consists of depth-scale measurements. To facilitate data processing, depths hav
1`. Please use negative depths (i.e. heights) to specify the priors on `alpha` parameters.
```

Your template may look different if you partitioned the sections differently.

Now we need to fill this template. Based on the correlation of Figure 1, we can assume that the oldest parts of the sections date from the upper or uppermost Permian. We specify that information via the priors on the $\alpha$ (shift) parameters. Note that $\alpha$ needs to be specified on the age scale

The $\gamma$ parameters are sedimentation rates, and we can use the dates to get vague estimates. Here is an example:

For the Changhsingian part of the Staithes section we can take the duration of the Changhsingian (~2.24 Myr) and the depth that these lithologies span to get a sedimentation rate estimate:

$$\frac{400\,\mathrm{m}}{2.24\,\mathrm{Myr}} \approx 180\frac{\mathrm{m}}{\mathrm{Myr}}$$

We take $180\frac{\mathrm{m}}{\mathrm{Myr}}$ as the mean for the prior distribution on $\gamma_{BunterShale}$ and $\gamma_{BunterSst}$. For the standard deviation, which should indicate our prior uncertainty of this estimate, we provisionally use $30\frac{\mathrm{m}}{\mathrm{Myr}}$. As we have to specify the priors for the logarithm of those parameters, we use the helper function `NormalToLogNormal(180, 30)` to give us the mean and standard deviation of a normal distribution on $\log\gamma_{BunterShale}$ and $\log\gamma_{BunterSst}$ that results in a mean of 180 and a standard deviation of 30 when raised to the power of $e$:

```r
NormalToLogNormal(180, 30)
```

```
$mu
[1] 5.179257

$sigma
[1] 0.1655264
```

The gaps, like the $\alpha$, need to be specified on the age scale.

Step 7: Fill the template for the priors with appropriate values.

Hint alpha

Let's fill the beginning of the template together.
For the $\alpha$ of the Staithes section, i.e. the section bottom, we can assume a roughly lower Changhsingian age, for example something between 252 and 255 Ma. For the North Sea section we might go a bit wider as the lowermost ZS5 unit extends further down there. We might go for a range of 252 to 260 Ma.
The top of the priors template can now be filled like this:

```
priors <- structure(list(
  `alpha_Staithes` = UniformPrior(min = 252, max = 255),
  `alpha_North Sea` = UniformPrior(min = 252, max = 260),
```

Hint gamma

Given the sedimentation rate example for earlier, we can calculate the mean and standard deviation for the prior on the Changhsingian sedimentation rate (in the Staithes section):

```
ChanghsingianMean <- NormalToLogNormal(mu = 180, sigma = 30)$mu
ChanghsingianSd <- NormalToLogNormal(mu = 180, sigma = 30)$sigma
```

For calculating sedimentation rates, we can also look up the depths of our litholgy (subsection) boundaries in the `lith` data.frame.
We can add this to the priors template:

```
priors <- structure(list(
  # placeholder for the alpha priors
  `gammaLog_Changhsingian` = NormalPrior(mean = ChanghsingianMean,
                                         sd = ChanghsingianSd),
```

For the sedimentation rate priors of the North Sea section, we could experiment with quite wide priors to see if the sulphur isotope data is enough to align it, or we can work from the correlation achieved in the previous worksheet or in Figure 1.

Hint gap

For the prior on the duration of the gaps in the Staithes section, we could take the duration of the Early Triassic (Induan and Olenekian), roughly 4.7 Myr. Let us place a normal prior with a mean of 4.7 Myr and a standard deviation of 0.5 Myr to allow for some uncertainty:

```r
priors <- structure(list(
  # placeholder for the alpha priors
  # placeholder for the gamma priors
  `gap_Staithes_1` = NormalPrior(mean  = 4.7, sd = 0.5),
```

We could take the same prior for the gap in the North Sea section.

Solution

Completing the template from what we already specified in the hints:
For the prior distribution on $\gamma_{AnisianLad}$ we take a normal distribution with $\frac{75\,\mathrm{m}}{5\,\mathrm{Myr}} = 14.5\frac{\mathrm{m}}{\mathrm{Myr}}$ as the mean, and a standard deviation of 3.
For the prior distribution on $\gamma_{C-N}$, we take $\frac{100\,\mathrm{m}}{28\,\mathrm{Myr}} = 3.8\frac{\mathrm{m}}{\mathrm{Myr}}$ as the mean, and a standard deviation of 0.5.
We transform the means and standard deviations of the normal distribution to get the values we can use for the normal priors on the $\log\gamma$:

```r
# Staithes
ChanghsingianMean <- NormalToLogNormal(mu = 180, sigma = 30)$mu
ChanghsingianSd <- NormalToLogNormal(mu = 180, sigma = 30)$sigma
AnisianLadMean <- NormalToLogNormal(mu = 14.5, sigma = 3)$mu
AnisianLadSd <- NormalToLogNormal(mu = 14.5, sigma = 3)$sigma
CNMean <- NormalToLogNormal(mu = 3.8, sigma = 0.5)$mu
CNSd <- NormalToLogNormal(mu = 3.8, sigma = 0.5)$sigma
```

For the priors on the sedimentation rate parameters of the North Sea section, we start with very vague priors, hoping that the sulphur isotopes are enough to align the sections and estimate the sedimentation rates. We use a uniform distribution, ranging from 3.8 (lowest sedimentation rate of Staithes) to 600 (three times the highest sedimentation rate of Staithes, as the sedimentation rate seems faster at the North Sea section). We can use $\log(3.8)$ and $\log(540)$ as the minimum and maximum of uniform priors on $\log\gamma$ of the North Sea section.

```r
myPriors <- structure(list(
  "alpha_Staithes" = UniformPrior(min = 252, max = 255),
  "alpha_North Sea" = UniformPrior(min = 252, max = 260),
  "gammaLog_Changhsingian" = NormalPrior(mean = ChanghsingianMean,
                                         sd = ChanghsingianSd),
  "gammaLog_Anisian-Lad" = NormalPrior(mean = AnisianLadMean,
                                       sd = AnisianLadSd),
  "gammaLog_C-N" = NormalPrior(mean = CNMean, sd = CNSd),
  "gammaLog_Z5" = UniformPrior(min = log(3.8), max = log(3*180)),
  "gammaLog_Bunter" = UniformPrior(min = log(3.8), max = log(3*180)),
  "gammaLog_Roet" = UniformPrior(min = log(3.8), max = log(3*180)),
  "gammaLog_M-Keuper" = UniformPrior(min = log(3.8), max = log(3*180)),
  "gammaLog_gap" = NormalPrior(mean = 4.7, sd = 0.5)),
  class = c("StratPrior", "list")
)
```

To check that these priors make sense, we can visualise the prior probability distributions.

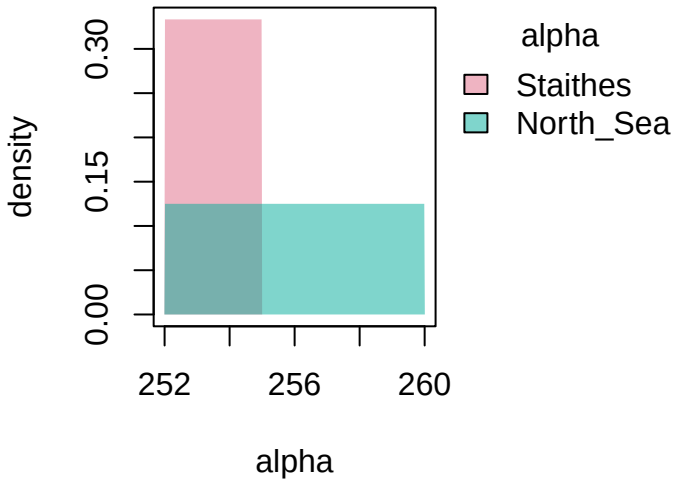
Step 8: Plot some of the prior distributions using the `plot.StratPrior()` function.

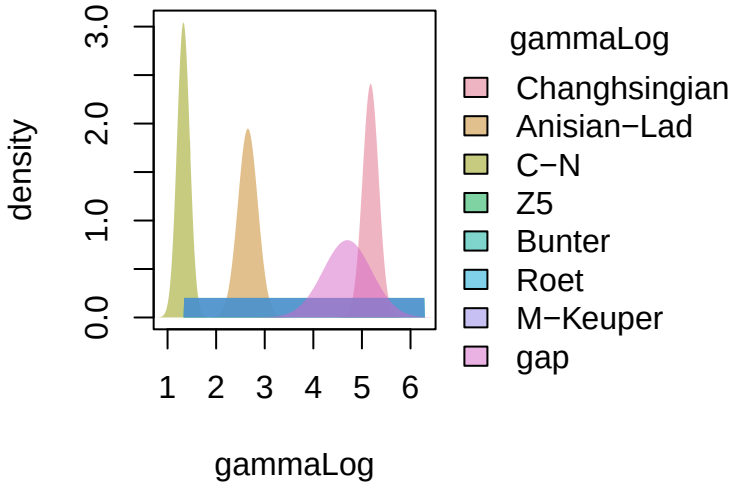
Hint

Specify the parameters for which you want to visualise the prior distributions by using the `params` argument, e.g. `params = c(1,3)`or `params = c("alpha_Staithes", "gammaLog_Changhsingian")`

Solution

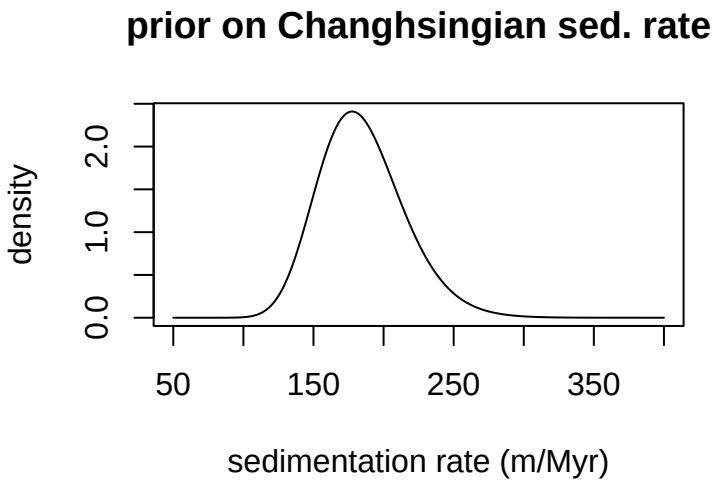
```r
plot(myPriors, params = c(1,3,10))
```




Plotting the probability density on the sedimentation rate scale, rather than on the log sedimentation rate scale, is not yet implemented in StratoBayes, but we could plot it manually like this:

```
rates <- seq(50,400,1)
dens <- dnorm(log(rates),ChanghsingianMean, ChanghsingianSd)
plot(rates, dens, type = "l", xlab = "sedimentation rate (m/Myr)",
     ylab = "density", main = "prior on Changhsingian sed. rate")
```




## Running the model

We now have everything we need to run the model. Here is what we will have to specify for the `RunStratModel()` function, along with some additional options:

Mandatory:

- `stratObject` The stratigraphic data read with the `StratData()` function.
- `alignmentScale` Specifying whether the model should run on the "height" or "age" scale
- `sedModel` The sedimentation rate model
- `prior` A `StratPrior` object with a structure given by the `PriorsTemplate()` function.

Often useful:

- `nKnots` Number of knots used for fitting the spline. The larger the number, the more accurately the spline can fit the data, but the slower the model will run.
- `nRun` Number of independent model runs that should be conducted.
- `nChains` Number of chains that should be run. The more chains, the more readily the model run can deal with complicated posterior distributions, but the slower it will run.
- `nIter` Number of iterations per model run
- `nThin` Factor by which the model output should be thinned – `nThin = 5` leads to only every 5th iteration being saved.

- `visualise` Iteration interval at which the current alignment should be visualised. Can be set to `FALSE` for no visualisation.
- `runParallel` Only relevant if `nRun > 1`. Runs independent model runs in parallel on different processors. If `visualise` is active, the alignment plots will be displayed in a browser window.
- `encourageOverlap` Alters the degree to which the model favours signal data overlapping each other, penalising a lack of overlap. We are still investigating which situations require adjustments to this penalty.
- `seed` Specify random seeds to make the model run reproducible.

It is advisable to do a test run with relatively few iterations, to check that it is working as intended.

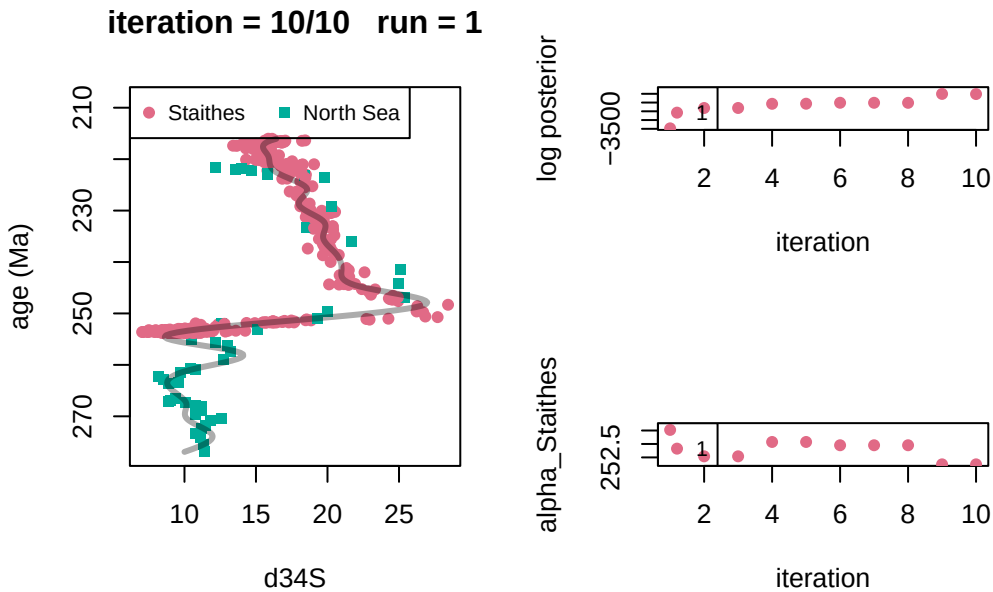
Step 9: Perform a test run using the `RunStratModel()` function.

Hint

Specify the arguments `data`, `scale`, `sedModel`, and `priors`.
Speed up the model run by using e.g. 100 iterations (`nIter = 100`) on 5 chains (`nChains = 5`), and set the number of knots to 20 (`nKnots = 20`).

Solution

```
# run model and save results to an R object named "result"
testRun <- RunStratModel(
  stratObject = stratDat,
  alignmentScale = "age",
  sedModel = "partition",
  priors = myPriors,
  nChains = 5,
  nIter = 10,
  nKnots = 20,
  seed = 10 # One seed -> one run
)
```




We used the random seed 10 (`seed = 10`) to generate consistent results when repeatedly running this code block, this is equivalent to the R command `set.seed(10)`).

Hopefully, this worked. Let's plot the test run to check that it's doing what intended.

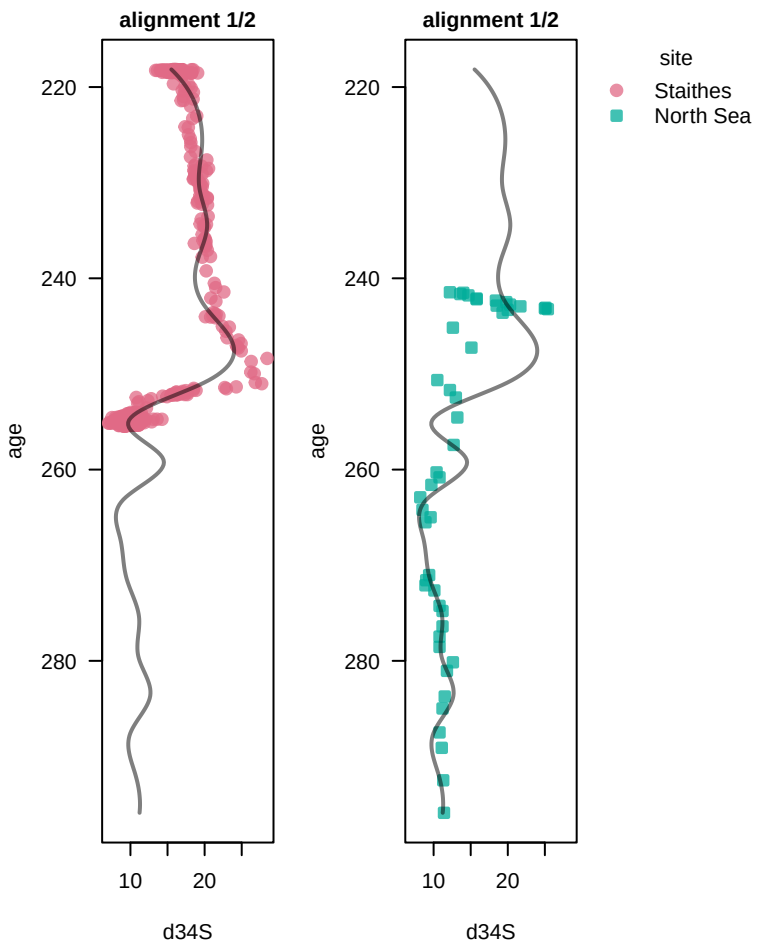
Step 10: Plot the test result using the `plot.StratPosterior()` function.

Hint

`plot()` will recognise that `testRun` is an object of class `StratPosterior`.

Solution

```
plot(testRun)
```




The alignment looks awful, but at least the Staithes section seems to already be roughly where the priors and age ties should push it.

We could do additional checks like traceplots here.

Step 11: Perform a longer run using the `RunStratModel()` function.

Hint

Specify the arguments `data`, `scale`, `sedModel`, and `priors`.
This time we want more iterations, e.g. 2000 iterations (`nIter = 2000`). We have used `20000` for the result shown later, but that might take a bit to run. Instead of 5 chains, let us use increase `nChains` to e.g. 12, and let use a larger number of knots (e.g. `nKnots = 30`). We can also try to run multiple runs in parallel by setting for example `nRun =`

4 and `runParallel = TRUE`. Control the interval of iterations at which the alignments should be plotted with `visualise`.

Solution

```
# run model and save results to an R object named "result"
result <- RunStratModel(stratObject = stratDat, alignmentScale = "age",
                        sedModel = "partition", priors = myPriors,
                        nChains = 12, nIter = 20000, nKnots = 30,
                        nRun = 4, seed = 1:4, runParallel = TRUE)
```

You can also load an existing model run using:

```
# load result for convenience
result <- readRDS("data/model-result-agemodel.rds")
```

## Looking at the results

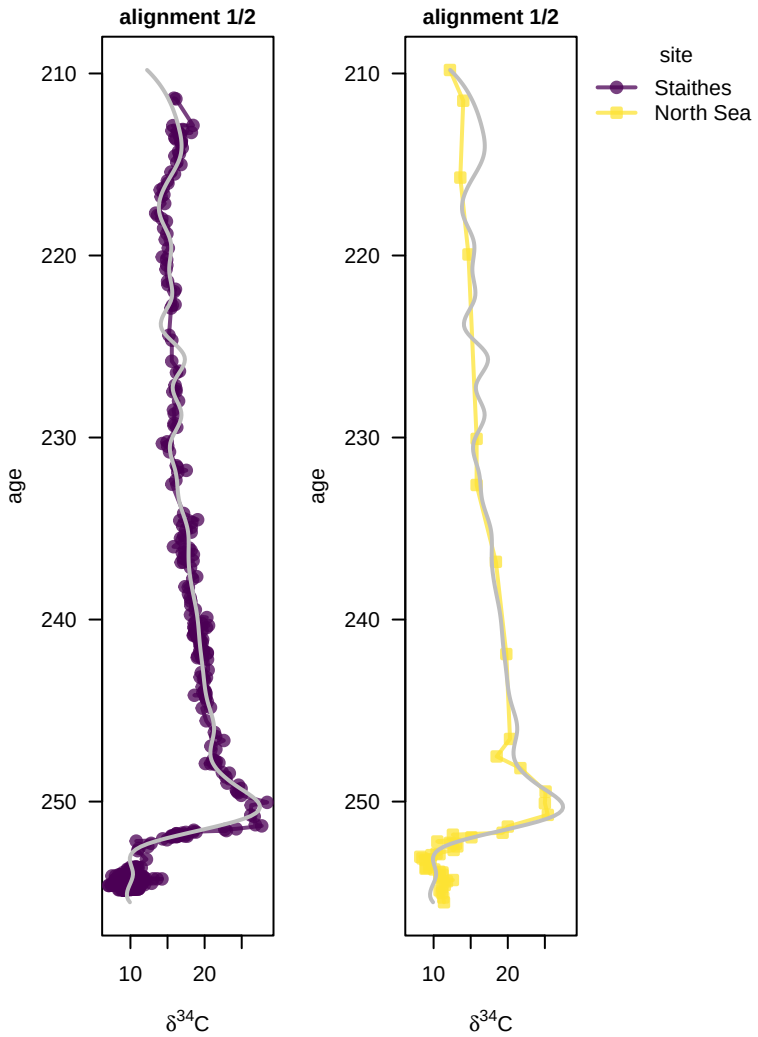
Step 12: Plot the result of the longer model run.

Hint

Again, use `plot()` or `plot.StratPosterior`. You can adjust the plot with graphical parameters, e.g. `xlab` for x-axis labels. There are a few custom graphical parameters which you can find out about using `?plot.StratPosterior`, e.g. the `colourScale` and the `splineColour` parameters.
If we want to look at the 25th instead of the 50th percentile of ages of the alignment, we can specify `percentile = 25`.

Solution

```
plot(result, type = "o", xlab = expression(delta^34*C),
     colourScale = "Viridis", splineColour = "grey", percentile = 25)
```




It seems the cluster analysis didn't find more than one distinct alignment in this example.

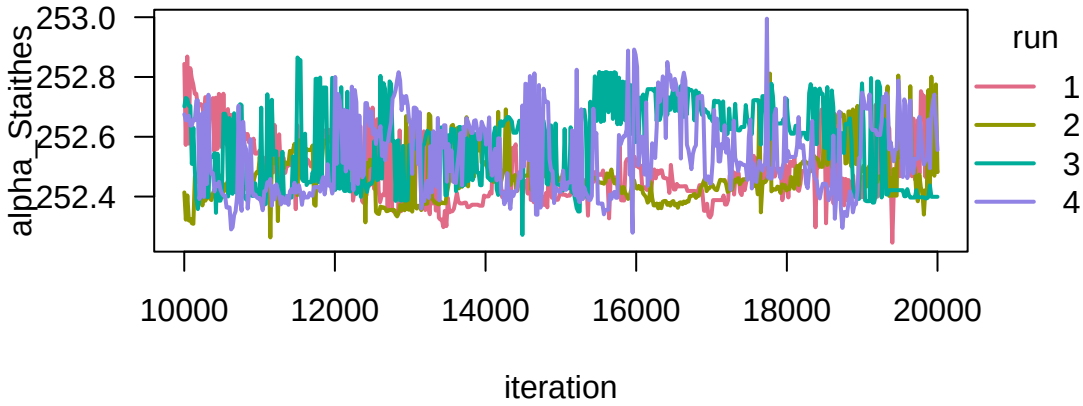
**Step 13: Inspect traceplots**

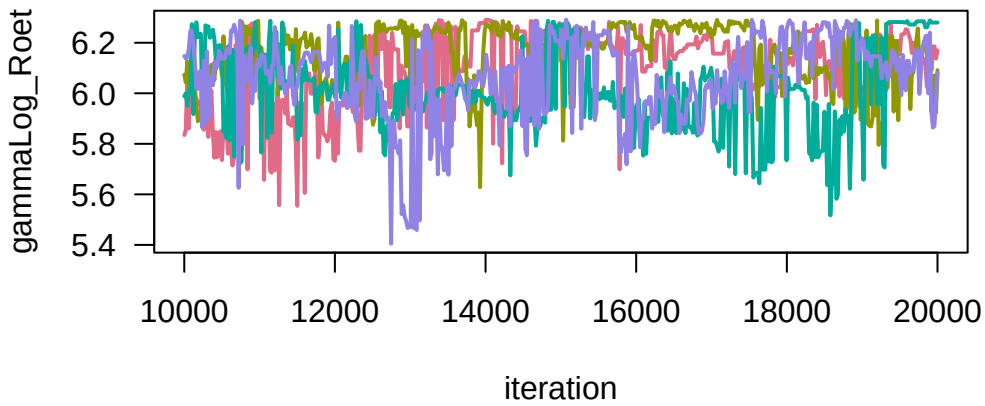
**Hint**

Again, use `TracePlot()`. You will need to provide the model parameters you want to inspect with the `parameters` argument, either as indices or as the parameter names saved in `result$model$paramsNames`.

Solution

```
TracePlot(result, parameters = c(1,8))
```




We should also inspect traceplots of the other model parameters, but these traceplots look like the MCMC may have converged, and if we let the model run for long enough, we should get robust estimates of the parameters.
Notice that per default, the first 50% of iterations are discarded as burn-in in all plot functions and other functions that deal with the model result.

Step 14: Get an age estimate for a depth of 2500 m in the North Sea section

Hint

This can be done using the `StratMap()` function. Make sure to specify the `site`, either with an index or by name. The depths need to be specified as negative heights, i.e. -2500.

Solution

```r
StratMap(result, heights = -2500, site = "North Sea")
```

```
  height     mean        sd     2.5%      50%    97.5%
1  -2500 253.3088 0.2351856 252.9965 253.2673 253.8845
```